

Access Control Policy

Owner: Mark Stolte, Founder — markstolte02@gmail.com

Last reviewed: 2026-07-05

Stage: Pre-launch / active development

Purpose

Defines who and what can access Penny Pilot's production systems and user data, and how that access is enforced technically rather than only by policy.

Two distinct access surfaces

Penny Pilot has two access surfaces that are easy to conflate and are deliberately kept separate:

1. **End-user access to their own data** — enforced in the database itself.
2. **Operator/admin access to the systems that run the app** — enforced by dashboard accounts at each vendor (GitHub, Supabase, Plaid, Apple, Google, Expo).

1. End-user access (in-product)

Every user-owned table in the Supabase schema (`profiles`, `setup_preferences`, `plaid_items`, `bank_accounts`, `categories`, `subcategories`, `transactions`, `merchant_rules`, `budgets`, `budget_lines`, `goals`, `forecast_snapshots`) has Row Level Security enabled, with a policy of the form `user_id = auth.uid()` (see `supabase/migrations/0001_initial_schema.sql`). This means:

- A user's Supabase session can only ever read or write rows where `user_id` matches their own authenticated id — enforced by Postgres, not by application logic, so a bug in the client or the API layer cannot leak another user's rows.
- The `entitlements` table is readable by its owner but writable only by trusted backend functions (no client-side insert/update policy exists for it), since entitlement state should not be user-editable.
- Plaid access tokens (`plaid_items.access_token_ciphertext`) are covered by the same per-user RLS policy for reads, but are additionally encrypted (see [Encryption Standards](#)) so that even a legitimate row read returns ciphertext, not a usable Plaid token.

2. Operator/admin access (infrastructure)

System	Current access holder(s)	Notes
--------	--------------------------	-------

GitHub org/repo (Mstolte02/penny-pilot)	Mark Stolte	Private repository.
Supabase project	Mark Stolte	Holds project owner role; can read Edge Function secrets, run migrations, and query the database directly.
Plaid dashboard	Mark Stolte	Currently Sandbox environment only; production access will be requested from Plaid before real bank data is synced.
Apple Developer Program / App Store Connect	Mark Stolte	Controls the com.pennypilot.finance bundle ID and Sign in with Apple configuration.
Google Cloud Console (OAuth client) / Play Console	Mark Stolte	Controls the Google Sign-In OAuth client.
Expo/EAS account (mstolte02s-team)	Mark Stolte	Controls builds and OTA updates.

Because there is currently one person, there is no separation-of-duties control between "writes the code" and "approves the deploy" — that gap is acknowledged rather than papered over, and is the first control introduced once a second person joins (see [Employee Offboarding](#) for the mirrored onboarding/offboarding checklist that will apply then).

Principle of least privilege for future access

When Penny Pilot adds a second person (employee, contractor, or agency), the default grant is the minimum needed for their function:

- Engineers: GitHub write access, a non-owner Supabase role scoped to the environments they work in, Sandbox-only Plaid dashboard access.
- Support/operations (if ever added): read-only Supabase dashboard access, no Edge Function secret visibility, no Plaid dashboard access.
- No one other than the founder holds the Plaid production secret, the Plaid token-encryption key, or the Supabase service-role key without a documented, time-boxed reason.

Service-role and secret handling

- The Supabase **anon key** is public by design (it ships in the client) and relies entirely on RLS for protection — this is the intended Supabase model, not a leak.
- The Supabase **service-role key**, the Plaid **client secret**, and the `PLAID_TOKEN_ENCRYPTION_KEY` are Supabase Edge Function secrets only. They are never placed in `EXPO_PUBLIC_*` variables, never bundled into the app, and never committed to git (`.env` is git-ignored; `.env.example` holds placeholders only).

Review cadence

Access is reviewed whenever a person's role changes, whenever a vendor account is added or removed, and at minimum annually as part of the [Risk Assessment Process](#).